

Laboratorio Búsquedas, Ordenamientos y Complejidad

Sergio Andres Rios Suarez

Parte A:

Parte B:

Para la colección de registros utilizamos un array para que el usuario registre los datos , los cuales son : Cantidad de dinero gastada durante 15 dias , el usuario utiliza la matriz para sus registros y asi tiene una mejor forma de organizar sus gastos

gastos = [[15000, 8000, 22000, 12000, 18000, 5000, 25000, 10000, 16000, 7000, 20000, 14000, 9000, 18000, 11000]]

son

Casos	# de comparaciones (n = cantidad de datos insertados)	Búsqueda secuencial	Busqueda binaria
Primer elemento	1 en secuencial n en binaria	1 ya que como es lineal compara los indices con el numero a encontrar teniendo solo esta comparacion	$\log_2(n)$ ya que divide el arreglo en la cantidad de lementos en grupos para la busqueda del elemento
Elemento del medio	$n/2$ en secuencial 1 en binaria	siendo n la cantidad de elementos , el # de comparaciones sera $n/2$ de los datos registrados por el usuario	1 ya que como es la mitad del elemento sera su busqueda miuhco mas facil
Ultimo elemento	n en secuencial (log n)	siendo n la cantidad de elementos , el # de comparaciones sera $n/2$ de los datos registrados por el usuario	$\log_2(n)$ ya que divide el arreglo en la cantidad de lementos en grupos para la busqueda del elemento
No existe	n en secuencial	siendo n la	$\log_2(n)$ ya que

		cantidad de elementos , el # de comparaciones sera $n/2$ de los datos registrados por el usuario	divide el arreglo en la cantidad de elementos en grupos para la busqueda del elemento
--	--	--	---

Parte C:

Datos desordenados :

```

***** BUBBLE SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 54
comparaciones: 95

***** SELECTION SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 0
comparaciones: 105

***** INSERTION SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 0
comparaciones: 14

```

Datos ordenados :

```

***** BUBBLE SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 0
comparaciones: 14

***** SELECTION SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 0
comparaciones: 105

***** INSERTION SORT *****
vector ordenado: [5000, 7000, 8000, 9000, 10000, 11000, 12000, 14000, 15000, 16000, 18000, 18000, 20000, 22000, 25000]
intercambios: 0
comparaciones: 14

```

su diferencia se basa principalmente en los intercambios realizados y el número de comparaciones de cada método por lo tanto, al tener el arreglo ordenado, unos ordenamientos no pueden funcionar mejor y realiza menos trabajo que los otros

```

#Parte A :

#1. el que hace menos trabajo es el insert sort ya que su complejidad cuando la lista ya esta ordenada es de  $O(n)$  lo cual solo verifica que la lista que ya tiene ordenada se mantiene en orden y solo inserta los elementos necesarios en su posición correcta.

#2 Para saber un array que hace el peor trabajo con el quick sort tenemos que tener en cuenta su funcionalidad la cual selecciona como

```

```

pivote el primer o último elemento de la lista. En este caso, si el
array ya está ordenado o casi ordenado, el quick sort tendrá que hacer
muchas comparaciones y movimientos, lo que resulta en su peor
rendimiento.
# un ejemplo del array que hace el peor trabajo con el quick sort sería
uno ya ordenado, como:
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] también uno ordenado de manera
descendente como:
# [10, 9, 8, 7, 6, 5, 4, 3, 2, 1] pero lo importante aca es que se
escoja el array de primer elemento como pivote teniendo en cuenta
también la cantidad de datos que tenga el array de forma general.

#3. es mucho más fácil utilizar la búsqueda binaria para realizar ese
proceso ya que como divide las listas en partes iguales en cada paso,
reduce significativamente la cantidad de comparaciones necesarias para
encontrar un elemento en comparación con la búsqueda lineal.
# además que la lista al estar ordenada facilita la aplicación de la
búsqueda binaria, ya que se puede dividir y verificar de manera
eficiente. además la complejidad de la búsqueda binaria es de  $O(\log n)$ ,
lo que la hace mucho más eficiente que la búsqueda lineal en listas
grandes

#Parte C :
#1.Bubble Sort: Compara elementos a pares y los intercambia si están
desordenados. Con datos desordenados hace  $O(n^2)$  comparaciones e
intercambios; con datos ya ordenados reduce sus comparaciones a  $O(n)$  y
cero intercambios. ya que recorre la lista solo para verificar el orden
#2.Selection Sort: Buscamos el elemento menor y lo colocamos al
principio y realiza exactamente la misma cantidad de comparaciones
 $O(n^2)$  tanto en datos desordenados como ordenados, ya que no podemos
realizar ningún aprovechamiento del orden previo
#3.Insertion Sort: Construye la lista ordenada uno por uno. Con datos
ordenados es  $O(n)$  lo cual nos ayuda a ser mucho más eficientes mientras
que con datos desordenados requiere  $O(n^2)$  por los desplazamientos
masivos

```

Parte F:

la verdad yo no estoy de acuerdo ya que C# no existe para compilar, sino para verificar de forma automática y frecuente que cada cambio no rompe el proyecto y Python, al ser interpretado y dinámico, se beneficia especialmente de pruebas y entornos reproducibles. La diferencia es que su pipeline no tiene la fase de compilación (lint , pruebas , empaquetado , despliegue), mientras que el de C++ sí (análisis estático , compilar , pruebas , el artefacto . el despliegue).